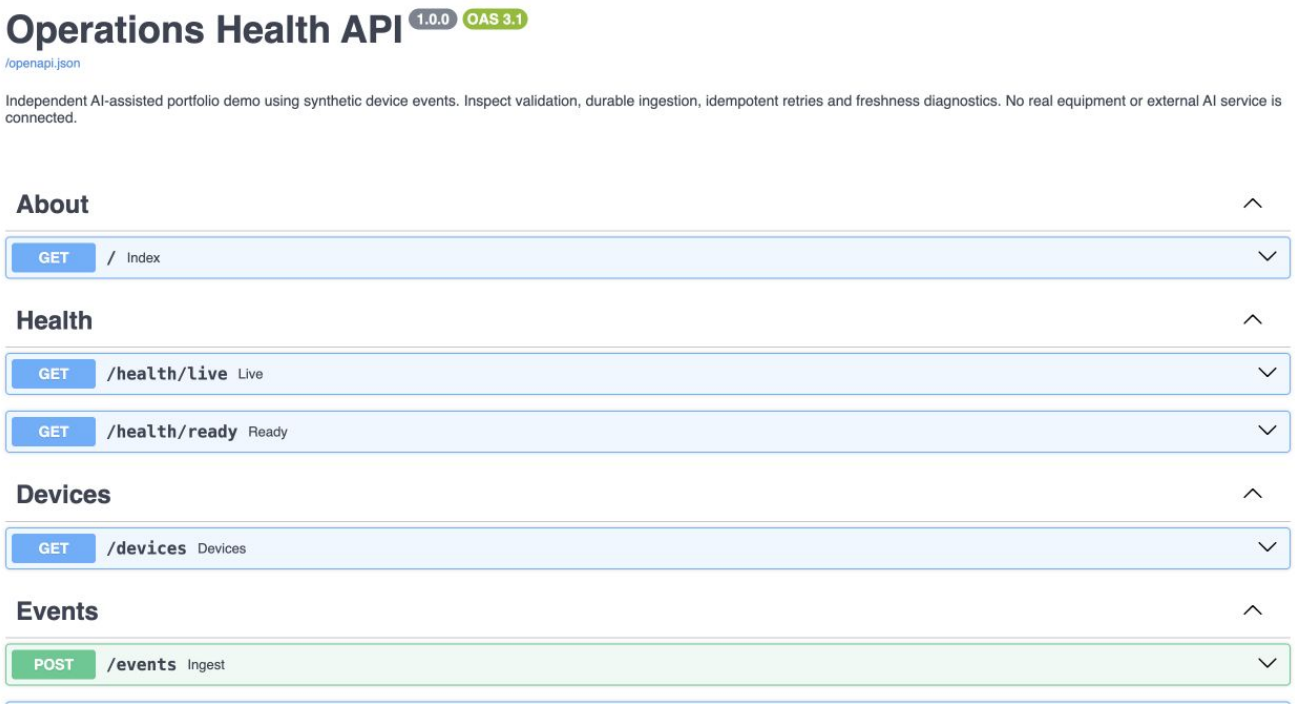
Operations Health API

FastAPI and Docker | Independent demonstration | Synthetic data

A small service for collecting device events and inspecting operational health. This portfolio demonstration was prepared with AI assistance using a generic use case; it contains no employer systems or data.

Scope and engineering choices

- Validate structured device events and return explicit input, device and conflict errors.
- Persist event history in SQLite; accept identical retries without adding duplicate rows.
- Filter and paginate history, and diagnose freshness, missing reports and clock skew.
- Separate liveness from storage readiness; include local-only Docker packaging.



API documentation from the locally running demonstration.

Why this is useful

The service makes input validation, event identity, storage and health checks easy to inspect. The source package includes reproducible setup steps and concrete failure scenarios, so a reviewer can check the behavior rather than rely on a feature list.

Verification and handover

Executed checks and clearly bounded deployment claims

Check	Observed outcome
Automated integration tests	15 passed, 0 failed. Includes eight concurrent retries, restart persistence, invalid inputs, out-of-order events and storage failure.
Live HTTP walkthrough	11 scenarios passed against the running localhost API. Observed 201, 200, 409, 422 and 404 responses are captured in JSON.
API documentation	Swagger UI opened in a browser against the live local service; screenshot supplied separately.
Compose configuration	docker compose config --quiet passed. This validates configuration syntax, not a running container.
Docker build and runtime	Not verified. The Docker client could not connect to a daemon; no image build or container startup was performed.

Reproduce the demonstration

- From backend-demo, create a Python 3.12 virtual environment and install requirements-lock.txt to match the tested package versions.
- Run: .venv/bin/python -m uvicorn app.main:app --host 127.0.0.1 --port 8765
- Open /docs, then run scripts/walkthrough.py against the local service. Run .venv/bin/python -m pytest -q for automated checks.

Delivery contents

Source code, pinned dependencies, configuration example, Docker build and Compose files, automated tests, captured execution evidence and a README with operational steps.

Scope limits

- Single-node synthetic demonstration with a fixed device registry; no real equipment, LLM provider, authentication or cloud deployment.
- Container files are supplied for review, with actual build and runtime validation still required. The Python base image tag is floating.
- One Starlette TestClient deprecation warning occurred with the pinned HTTPX version. All executed tests passed; the tested dependency closure is included.

A newly prepared independent sample demonstrates only the behavior actually inspected and tested. It does not represent prior customer acceptance or a production deployment.